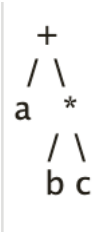
作业 HW3* 实验报告

姓名：李欣洋 学号：2453359 日期：2025 年 11 月 10 日

实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）1
实验报告要求在文字简洁的同时将内容表示清楚
报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景
实验整体使用到的数据结构是二叉树，其他辅助数据结构为栈 队列 顺序表。
题目一:二叉树的遍历可以通过非递归来实现，本题在给出二叉树的中序非递归遍历的情况下，可以唯一确定一棵二叉树，然后输出其后序遍历序列。
题目二:给定两棵树 T1 和 T2。如果 T1 可以通过若干次左右孩子互换变成 T2，则我们称两棵树是“同构”的。本题通过编写代码来确定给出的两棵树是否是同构的。
题目三:给定一棵二叉树，以及一个 start 值，将从值为 start 的结点开始感染，通过编写代码来确定感染一棵二叉树所需要的时间。
题目四:给出一棵树 DFS 遍历的方向，d 代表下行，u 代表上行，通过其 DFS 遍历的方向，将这一棵树改写为二叉树，并且输出改写前树的深度和改写后二叉树的深度。
题目五: 给出一颗多叉树，求出两个节点的最近公共祖先。
题目六: 给出二叉树的前序遍历和中序遍历，求树的后序遍历。
题目七:任何一个表达式，都可以用一棵表达式树来表示。例如，表达式 a+b*c，可以表示为如下形式。给出一个中缀表达式，这个中缀表达式用变量来表示（不含数字），通过编写代码将这个中缀表达式用表达式二叉树的形式输出出来。

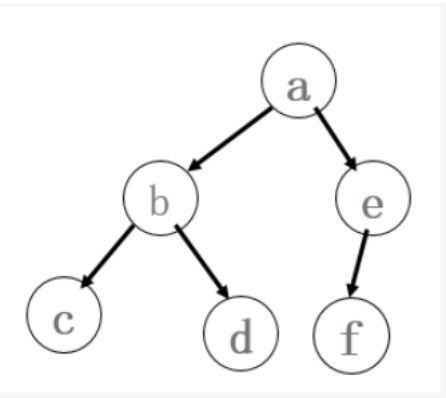


2. 实验内容

3.1 二叉树的非递归遍历

3.1.1 问题描述

二叉树的非递归遍历可通过栈来实现。例如对于由 abc##d##ef###先序建立的二叉树，如下图所示，中序非递归遍历，可以通过如下一系列栈的入栈出栈操作来完成：push(a) push(b) push(c) pop pop push(d)pop pop push(e) push(f) pop pop。



已知中序遍历的栈的操作序列，就可唯一地确定一棵二叉树。请编程输出该二叉树的后序遍历序列。

3.1.2 基本要求

简短地描述题目要求
输入:
第一行一个整数 n，表示二叉树的结点个数。
接下来 2n 行，每行描述一个栈操作，格式为：push X 表示将结点 X 压入栈中，pop 表示从栈中弹出一个结点。
(X 用一个字符表示)
输出:
一行，后序遍历序列。

3.1.3 数据结构设计

```
struct TreeNode
{
    char value;
    TreeNode* left;
    TreeNode* right;
    TreeNode(charval):value(val),left(nullptr),right(nullptr){};
};
```

3.1.4 功能说明（函数、类）

```
    //初始化一个栈用来还原二叉树
    stack<TreeNode*>Tree;
    //根节点
    TreeNode* root=nullptr;
    //遍历中临时存储被 pop 的节点
    TreeNode* last=nullptr;
```

```
    for(int i=0;i<2*n;i++)
    {
        //当遇到 operation 为 push 时，说明该结点是当前栈顶结点的左孩子或者
右孩子
        if(operation[i]=="push")
        {
            //建立一个新结点
            TreeNode* temp=new TreeNode(value[i]);
            //如果根节点为空，则将该结点设为根节点
            if(!root)root=temp;
            //在栈非空的情况下
            if(!Tree.empty())
            {
                //如果栈顶结点无左孩子，说明该结点是其左孩子
                if(!Tree.top()->left)
                  Tree.top()->left=temp;
                //否则该结点是其右孩子
                else
                  last->right=temp;
            }
            //特判，当栈为空时，说明 pop 了根节点，此时 last 指向的结点是根
节点
            if(last!=nullptr && Tree.empty())
              last->right=temp;
            Tree.push(temp);//入栈
        }
        else
        {
            //遇到 pop 操作时，让栈顶元素出栈，但此时需要 last 记录下被 pop
的节点，以便下一次 push 操作时能正确连接右孩子
            last=Tree.top();
            Tree.pop();
```

```
        }
    }
```

3.1.5 调试分析（遇到的问题和解决方法）

此题遇到的问题为，如何处理 pop 这一步之后的 push？即如何处理右孩子的插入逻辑。

我的第一版代码只使用了一个 root 指针来处理左孩子的插入逻辑，并没有考虑到再维护一个 last 指针来处理右孩子的插入逻辑。举个例子来说，输入 push a push b push c pop pop push d 这样的代码，a 的左孩子是 b，b 的左孩子是 c，两次 pop 之后，此时栈内仅存的结点是 a，但是紧接着又是一次 push d 操作，这样如果按照，当前栈顶结点的左孩子如果不为空，那么新加入的结点是其右孩子的逻辑，那么 d 会变成 a 的右孩子，但是按照题目要求来看，d 显然应该是 b 的右孩子。这样就和题目要求相矛盾。

所以我考虑维护了一个 last 指针，每一次 pop，就将 last 更新为最后一次 pop 的结点。这样在当前结点的左孩子不为空时，逻辑更改为 last 的右孩子指向新结点，即可解决这个问题。

3.1.6 总结和体会

这道题要求根据中序遍历的 push/pop 操作序列还原二叉树并输出后序遍历结果，算法难度较高。核心思路是利用栈来逆向构建树结构：通过判断栈顶节点是否有左孩子来决定新节点的位置，并用 last 指针记录被 pop 的节点以正确连接右子树。算法的时间和空间复杂度均为 O(n)，是线性高效的解法。实现中需要特别注意三个边界条件：根节点的初始化、栈为空时的特殊处理（根节点 pop 后连接右子树）、以及左右孩子的判断逻辑。通过这道题深入理解了栈在树遍历中的作用机制，掌握了从遍历序列反向构建二叉树的技巧，以及中序遍历操作与树结构的对应关系，对二叉树的非递归遍历和指针操作有了更深刻的认识。

3.2 二叉树的同构

3.2.1 问题描述

给定两棵树 T1 和 T2。如果 T1 可以通过若干次左右孩子互换变成 T2，则我们称两棵树是“同构”的。例如图 1 给出的两棵树就是同构的，因为我们把其中一棵树的结点 a、b、e 的左右孩子互换后，就得到另外一棵树。而图 2 就不是同构的。现给定两棵树，请你判断它们是否是同构的。并计算每棵树的深度。

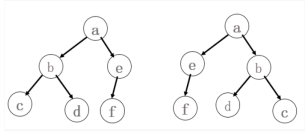


图1

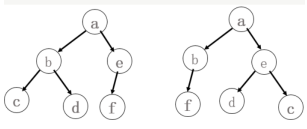


图2

3.2.2 基本要求

输入:

第一行是一个非负整数 N1，表示第 1 棵树的结点数；
随后 N 行，依次对应二叉树的 N 个结点（假设结点从 0 到 N−1 编号），每行有三项，分别是 1 个英文大写字母、其左孩子结点的编号、右孩子结点的编号。如果孩子结点为空，则在相应位置上给出“-”。给出的数据间用一个空格分隔。
接着一行是一个非负整数 N2，表示第 2 棵树的结点数；
随后 N 行同上描述一样，依次对应二叉树的 N 个结点。

输出:
共三行。
第一行，如果两棵树是同构的，输出“Yes”，否则输出“No”。
后面两行分别是两棵树的深度。

3.2.3 数据结构设计

```
struct TreeNode
{
    int left;//左孩子

    int right;//右孩子

    char data;//存储的值

    TreeNode():right(-1),left(-1){};
};
```

3.2.4 功能说明（函数、类）

```
bool JudIsomorphism(int root_1,int
root_2,vector<TreeNode>&T1,vector<TreeNode>&T2)
{
  //考虑使用递归函数来解决问题

  //root_1 和 root_2 分别是两棵树的根节点，T1 和 T2 分别是两棵树的存储结构

  //当左右对应的结点均空，则同构
    if(root_1==-1 && root_2==-1)
      return true;

  //一个为空一个非空，显然不同构
    if(root_1==-1 || root_2==-1)
      return false;

  //对应结点的值不同，显然不同构
```

```
    if(T1[root_1].data!=T2[root_2].data)
      return false;

  //分别获取左右孩子的下标

    int L1=T1[root_1].left;
    int R1=T1[root_1].right;
    int L2=T2[root_2].left;
    int R2=T2[root_2].right;

  //分别判断左右孩子交换的情况和不交换的情况

    bool swap=JudIsomorphism(L1,R2,T1,T2) &&
JudIsomorphism(R1,L2,T1,T2);
    bool noswap=JudIsomorphism(L1,L2,T1,T2) &&
JudIsomorphism(R1,R2,T1,T2);

  //只要交换或不交换中的一种情况成立就说明两棵树是同构的

    return swap || noswap;
}
```

3.2.5 调试分析（遇到的问题和解决方法）

在此题中遇到的第一个问题即，找到这棵树的根节点。因为采取结点数组的方式进行存储，所以只需要再开一个 bool 数组，然后遍历结点数组，当遇到某一结点的 left 或 right 的值不为-1 的情况下，将 bool 数组中相同位置的元素置为 false，然后遍历 bool 数组返回值为 true 的位置，即根节点的位置。
之后考虑使用递归的方法来判断两棵树是否同构比较显然，只需要注意需要判断交换与不交换两种情况，而两种情况只需要满足其一即可。

3.2.6 总结和体会

这道题要求判断两棵二叉树是否同构（可通过交换子树得到相同结构），算法难度中等偏上。核心思路是使用递归函数同时检查子树交换和不交换两种情况，只要其中一种成立即判定为同构。算法时间复杂度为 O(n)，包括 FindRoot 寻找根节点、JudIsomorphism 递归判断同构、以及 getDepth 计算深度三个主要部分。实现中需要注意几个关键边界条件：双空返回 true、单空返回 false、节点值不同返回 false、节点个数不同直接判定不同构。易错点主要有三个：一是同构判断时 swap 和 noswap 必须是"或"关系而非"与"关系；二是 FindRoot 需要正确标记所有子节点来找出根节点；三是输入处理时要用 cin.ignore()清除换行符，并正确处理"-"表示的空节点。

3.3 感染二叉树需要的总时间

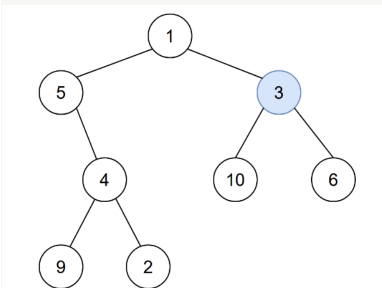
3.3.1 问题描述

给你一棵二叉树的根节点 root ，二叉树中节点的值 **互不相同** 。另给你一个整数 start 。在第 0 分钟，**感染** 将会从值为 start 的节点开始爆发。
每分钟，如果节点满足以下全部条件，就会被感染：

- 节点此前还没有感染。

- 节点与一个已感染节点相邻。

返回感染整棵树需要的分钟数。**例如下图中** start = 3，**输出：** 4



3.3.2 基本要求

输入:
第一行包含两个整数 n 和 start
接下来包含 n 行，描述 n 个节点的左、右孩子编号
0 号节点为根节点，0<=start<n

输出:
一个整数，表示感染整棵二叉树所需要的时间

3.3.3 数据结构设计

```
struct TreeNode
{
    int left;
    int right;
    TreeNode():left(-1),right(-1){};
};
```

3.3.4 功能说明（函数、类）

```
int BFS(int n,vector< vector<int> >&G,int start)
{

    //Translate 的辅助函数，使用 BFS 遍历无向图，计算感染所需时间

    int count=0;

    //visited 记录该结点是否被感染过

    vector<bool>visited(n,false);

    //队列用于 BFS 遍历

    queue<int>q;

    //起始感染结点入队
```

```
    q.push(start);
    visited[start]=true;

    while(!q.empty())
    {
        //Jud 用于判断本层是否有新节点被感染，有的情况下才增加时间
        bool Jud=false;
        int level=q.size();
        for(int i=0;i<level;i++)
        {
            int Tree=q.front();
            q.pop();
            for(int val : G[Tree])
            {
                if(!visited[val])
                {
                    //该结点未被感染过，则感染该结点，且入队
                    visited[val]=true;
                    q.push(val);
                    Jud=true;
                }
            }
        }
        //本层有新节点被感染，时间加 1
        if(Jud) count++;
    }
    return count;
}
int Translate(vector<TreeNode>&T,int start)
{
    //将此二叉树转换为无向临接表
    vector<vector<int>>graph(T.size());
    for(int i=0;i<T.size();i++)
    {
        if(T[i].left!=-1)
        {
            //当前结点的邻居加入左孩子，左孩子的邻居加入当前结点
            graph[i].push_back(T[i].left);
            graph[T[i].left].push_back(i);
```

```
        }
        if(T[i].right!=-1)
        {
            //当前结点的邻居加入右孩子，右孩子的邻居加入当前结点
            graph[i].push_back(T[i].right);
            graph[T[i].right].push_back(i);
        }
    }
    //使用 BFS 遍历无向图，计算感染所需时间
    //为何使用 BFS？因为感染时间一定是最短的，而 BFS 恰好能找到最短路径
    return BFS(T.size(),graph,start);
}
```

3.3.5 调试分析（遇到的问题和解决方法）

此题最难的问题在于算法的选取。如何能够高效的完成对一整棵树的感染而不重复访问结点。我们注意到，此题的感染时间一定为最短时间，且被感染过的结点并不会被重复感染。所以我们很容易想到使用 BFS 来解决这个问题。

将此二叉树改用无向临接表来存储，即父结点的邻居存储孩子，同时孩子的邻居也存入父亲。之后从起始结点开始，运用 BFS 即可解决。唯一需要额外注意的是，需要加入一个 bool 类型的变量来判断这一次的遍历有没有感染新的结点，只有在感染了新的结点的情况下，感染的时间才加 1。

3.3.6 总结和体会

这道题要求计算二叉树从某个起始节点开始感染所有节点所需的时间，算法难度中等。核心思路是将二叉树转换为无向图（因为感染可双向传播），然后使用 BFS 按层遍历模拟感染扩散过程，每传播一层时间增加 1。算法时间和空间复杂度均为 O(n)，构建邻接表和 BFS 遍历都是线性时间操作。实现中需要注意几个关键边界条件：左右孩子为-1 时不建边、起始节点立即标记已访问、使用 Jud 标志判断当前层是否真正有新感染节点发生。易错点主要有三个：一是必须建立无向边，即双向添加 i->child 和 child->i，否则无法从子向父传播；二是时间计数不能简单用 BFS 层数，而要判断每层是否真正有新节点被感染；三是 visited 数组必须正确初始化和更新，防止节点重复访问。通过这道题深入理解了树到图的转换思想，掌握了 BFS 在模拟扩散类问题中的应用技巧，学会了使用 queue 的 size()方法实现分层遍历，以及如何用布尔标志精确控制计数逻辑，对图的遍历算法和实际问题建模有了更深刻的认识。

3.4 树的重构

3.4.1 问题描述

树木在计算机科学中有许多应用。也许最常用的树是二叉树，但也有其他的同样有用的树类型。

其中一个例子是有序树（任意节点的子树是有序的）。
每个节点的子节点数是可变的，并且数量没有限制。 一般而言， 有序树由有限节点集合 T 组成，并且满足：

1.其中一个节点置为根节点，定义为 root(T);

2.其他节点被划分为若干子集 T1,T2,...Tm,每个子集都是一个树.

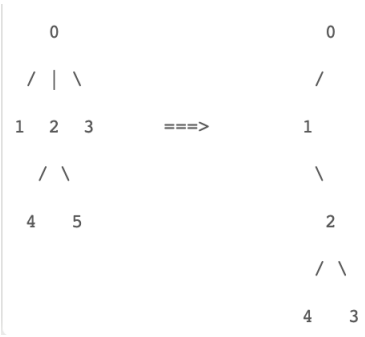
同样定义 root(T1),root(T2),...root(Tm)为 root(T)的孩子， 其中 root(Ti)是第 i 个孩子。节点 root(T1),...root(Tm)是兄弟节点。
通常将一个有序树表示为二叉树是更加有用的，这样每个节点可以存储在相同内存空间中。有序树到二叉树的转化步骤为：

1.去除每个节点与其子节点的边

2.对于每一个节点，在它与第一个孩子节点（如果存在）之间添加一条边，作为该节点的左孩子

3.对于每一个节点，在它与下一个兄弟节点（如果存在）之间添加一条边，作为该节点的右孩子

如图所示：
在大多数情况下，树的深度（从根节点到叶子节点的边数的最大值）都会在转化后增加。这是不希望发生的事情因为很多算法的复杂度都取决于树的深度。实现一个程序来计算转化前后的树的深度。



3.4.2 基本要求

输入:
输入由多行组成，每一行都是一棵树的深度优先遍历时的方向。其中 d 表示下行(down)，u 表示上行(up)。
例如上面的树就是 dudduduudu,表示从 0 下行到 1,1 上行到 0,0 下行到 2 等等。输入的截止为以#开始的行。
输出：
对每棵树，打印转化前后的树的深度，采用以下格式 Tree t: h1 => h2。其中 t 表示样例编号(从 1 开始)，h1 是转化前的树的深度，h2 是转化后的树的深度。

3.4.3 数据结构设计

```
struct TreeNode
{
    int left_child;
    int right_brother;
```

```
    int last_child;//记录当前节点的最后一个孩子节点，方便插入新的兄弟节点
    TreeNode():left_child(-1),right_brother(-1),last_child(-1){};
};
```

3.4.4 功能说明（函数、类）

```
//循环读取每一行 DFS 遍历中的'u'和'd'
        for(char c : line)
        {
            //父结点是栈的顶元素
            int parent=s.top();
            if(c=='d')
            {
                //如果操作是下行，则先将 count++，代表新结点的编号
                count++;
                //如果父结点的左孩子为空，则将新结点设为左孩子
                //否则将新结点设为当前父结点最后一个孩子的右兄弟
                if(Tree[parent].left_child==-1)
                {
                    Tree[parent].left_child=count;
                }
                else
                {
Tree[Tree[parent].last_child].right_brother=count;
                }
                //更新父结点的最后一个孩子为当前新结点
                Tree[parent].last_child=count;
                s.push(count);
                //更新树的高度
                h1=max(h1,(int)s.size()-1);
            }
            else if(c=='u')
            {
                //如果操作是上行，则将栈顶元素弹出
```

```
            if(s.size() > 1)
                s.pop();
        }
    }

    }
```

3.4.5 调试分析（遇到的问题和解决方法）

此题遇到的问题就是在于如何处理由一棵二叉树 DFS 遍历 d 和 u 组成的字符串。遇到 d 的时候，将当前父结点的左孩子置为新结点即可，这里不难想到。但是遇到 u 之后的 d，应该做出特殊处理。
在每一次下行之后，代表此时 DFS 遍历上行，我需要维护当前父结点的 lastchild，即最后一个孩子，指向此时新加入的结点，在后序要到 d 的时候，如果当前父结点的左孩子不为-1，此时应该让父结点的 lastchild 的右兄弟指向新结点。

3.4.6 总结和体会

这道题要求根据 DFS 遍历操作序列重构多叉树，并将其转换为孩子-兄弟表示法的二叉树，然后比较转换前后的树高度，算法难度较高。核心思路是使用栈模拟 DFS 过程：遇到'd'操作创建新节点并入栈，遇到'u'操作弹栈回溯，通过栈顶元素维护当前父节点。关键数据结构是 left_child（左孩子）、right_brother（右兄弟）和 last_child（最后一个孩子）三个指针，其中 last_child 用于快速定位兄弟链末尾以插入新兄弟。算法时间和空间复杂度均为 O(n)，包括构建树和计算高度两个线性过程。实现中需要注意几个边界条件：空行跳过、'#'结束、栈保护（size>1 才能 pop）、首个孩子与后续兄弟的不同处理逻辑。易错点主要有三个：一是 last_child 必须在每次添加孩子后及时更新，否则兄弟链会断裂；二是'u'操作时必须判断栈大小，防止弹出根节点；三是原树高度用栈最大深度-1 计算，而转换后二叉树需要同时遍历左孩子和右兄弟两个维度。

3.5 最近公共祖先

3.5.1 问题描述

给出一颗多叉树，请你求出两个节点的最近公共祖先。
一个节点的祖先节点可以是该节点本身，树中任意两个节点都至少有一个共同祖先，即根节点。

3.5.2 基本要求

输入:
输入数据包含 T 个测试样本，每个样本 i 包含 Ni 个节点和 Ni-1 条边和 Mi 个问题，树中节点从 1 到 Ni 编号
输入第一行是测试样本数 T
每个测试样本 i 第一行为两个整数 Ni 和 Mi
接下来 Ni-1 行，每行 2 个整数 a、b，表示 a 是 b 的父节点
接下来 Mi 行，每行两个整数 x、y，表示询问 x 和 y 的共同祖先

输出:
对于每一个询问输出一个整数，表示共同祖先的编号

3.5.3 数据结构设计

```
    //用临接矩阵存储结点的孩子
    vector<vector<int>>children(N+1);
    //用数组存储结点的父节点
    vector<int>parent(N+1);
    //辅助数组，判断某节点是否有父节点，便于后续寻找根节点
    vector<bool>HasParent(N+1,false);
```

3.5.4 功能说明（函数、类）

```
int FindAncestor(const vector<int>& parent, int x, int y)
{
    //x 和 y 分别为需要查询的两个结点

    //使用 visited 数组记录从 x 到根节点的路径上的所有父结点
    vector<bool> visited(parent.size(), false);
    int cur = x;
    while (cur != 0)
    {
        //将此时的结点标记为已访问
        visited[cur] = true;
        //更新当前结点为其父节点
        cur = parent[cur];
    }

    cur = y;
    while (cur != 0)
    {
        //当 y 的某个父节点在 x 的路径上出现时，说明该节点为公共祖先，返回其值
        if (visited[cur]) return cur;
        //更新当前结点为其父节点
        cur = parent[cur];
    }
    return -1;
```

```
}
```

3.5.5 调试分析（遇到的问题和解决方法）

此题在编写时一次就想到了思路并且成功完成了实现。
遇到的问题很直接，就是怎么判断两个结点找到了公共祖先。在思考之后就想到了思路，遍历 parent 数组，同时开一个大小相同的 bool 数组，从初始结点开始，每一次遇到父结点就把和下标为父结点的 bool 数组对应的元素改为 true。然后遍历到根节点结束。
之后从第二个结点开始遍历，当第二个结点的遍历过程中，某一个父结点对应的 bool 数组的值为 true 时，即说明找到了公共祖先，返回其值即可。

3.5.6 总结和体会

这道题要求在多组测试数据中查询树中两个节点的最近公共祖先(LCA)，算法难度中等。核心思路是使用路径标记法：先从节点 x 向上遍历到根并用 visited 数组标记路径上所有祖先，再从节点 y 向上遍历，第一个遇到的已标记节点即为 LCA。算法时间复杂度为 O(T(N+Mh))，包括构建树 O(N)、寻找根节点 O(N)和 M 次查询每次 O(h)；空间复杂度为 O(N)用于存储父节点数组和访问标记。实现中需要注意几个边界条件：根节点 parent 设为 0 作为循环终止标志、使用 HasParent 数组找到无父节点的根、特判 x 或 y 本身是根的情况可直接返回根节点。易错点主要有三个：一是根节点 parent 值必须与 while 循环条件一致（都用 0），否则会死循环或提前终止；二是 visited 数组大小必须是 parent.size()即 N+1，因为节点编号从 1 开始；三是容易遗漏根节点特判，虽然不影响正确性但会浪费不必要的遍历。

3.6 求树的后序

3.6.1 问题描述

给出二叉树的前序遍历和中序遍历，求树的后序遍历

3.6.2 基本要求

输入:
输入包含若干行，每一行有两个字符串，中间用空格隔开
同行的两个字符串从左到右分别表示树的前序遍历和中序遍历，由单个字符组成，每个字符表示一个节点
字符仅包括大小写英文字母和数字，最多 62 个
输入保证一颗二叉树内不存在相同的节点

输出:
每一行输入对应一行输出
若给出的前序遍历和中序遍历对应存在一棵二叉树，则输出其后序遍历
否则输出 Error

3.6.3 数据结构设计

```
//两个 string 变量分别存储前序和中序遍历结果
string Preorder,Inorder;
```

3.6.4 功能说明（函数、类）

```
int FindAncestor(const vector<int>& parent, int x, int y)
{
    //x 和 y 分别为需要查询的两个结点

    //使用 visited 数组记录从 x 到根节点的路径上的所有父结点
    vector<bool> visited(parent.size(), false);
    int cur = x;
    while (cur != 0)
    {
        //将此时的结点标记为已访问
        visited[cur] = true;
        //更新当前结点为其父节点
        cur = parent[cur];
    }

    cur = y;
    while (cur != 0)
    {
        //当 y 的某个父节点在 x 的路径上出现时，说明该节点为公共祖先，返回其值
        if (visited[cur]) return cur;
        //更新当前结点为其父节点
        cur = parent[cur];
    }
    return -1;
}
```

3.6.5 调试分析（遇到的问题和解决方法）

这道题目使用递归的思路来解决比较显然。但是在我第一次编写代码时，最后得到的答案永远是Error。后来我发现问题出在：必须在递归返回后检查 leftPost 和 rightPost 是否为"#"，任一子树错误则整棵树错误。
第二个问题就是，子串划分的边界计算：leftPre 应该从位置 1 开始长度为 index（不包括根），

rightPre 从位置 1+index 开始到末尾；leftIn 从位置 0 开始长度为 index，rightIn 从位置 index+1 开始。

3.6.6 总结和体会

这道题要求根据二叉树的前序和中序遍历结果推导出后序遍历，算法难度中等。核心思路是利用前序首元素是根、中序中根左侧是左子树右侧是右子树的性质，通过递归分治不断分解问题：找到根在中序中的位置 index，据此划分左右子树的前序和中序序列，递归求解后按"左右根"顺序拼接结果。算法时间复杂度为 $O(n^2)$, 因为每层递归都要 find()查找 O(n)和 substr()创建子串 O(n)；空间复杂度也是 $O(n^2)$，每层递归创建新字符串。可通过传递索引替代子串创建优化至 O(n)。实现中需要注意几个边界条件：空串返回空串作为递归终止、find()返回 npos 时说明输入不合法返回"#"、任一子树返回"#"则向上传播错误。易错点主要有三个: 一是子串边界计算必须精确, leftPre 从 1 开始长度 index，rightPre 从 1+index 到末尾，leftIn 从 0 开始长度 index，rightIn 从 index+1 到末尾；二是必须检查递归返回的"#"标记并向上传播，保证错误能被正确处理；三是必须判断 find()是否返回 npos，否则当前序中序不匹配时会导致程序异常。

3.7 表达式树

3.7.1 问题描述

任何一个表达式，都可以用一棵表达式树来表示。例如，表达式 a+b*c，可以表示为如下的表达式树:

```
  +
 / \
a   *
   / \
   b c
```

现在，给你一个中缀表达式，这个中缀表达式用变量来表示（不含数字），请你将这个中缀表达式用表达式二叉树的形式输出出来。

3.7.2 基本要求

输入:
输入分为三个部分。
第一部分为一行，即中缀表达式(长度不大于 50)。
中缀表达式可能含有小写字母代表变量（a-z），也可能含有运算符（+、-、*、/、小括号），不含有数字，也不含有空格。
第二部分为一个整数 n(n <= 10)，表示中缀表达式的变量数。
第三部分有 n 行，每行格式为 C　x，C 为变量的字符，x 为该变量的值。

输出:
输出分为三个部分，第一个部分为该表达式的逆波兰式，即该表达式树的后根遍历结果。占一行。
第二部分为表达式树的显示，如样例输出所示。

如果该二叉树是一棵满二叉树，则最底部的叶子结点，分别占据横坐标的第 1、3、5、7……个位置（最左边的坐标是 1），
然后它们的父结点的横坐标，在两个子结点的中间。
如果不是满二叉树，则没有结点的地方，用空格填充（但请略去所有的行末空格）。
每一行父结点与子结点中隔开一行，用斜杠（/）与反斜杠（\）来表示树的关系。
/出现的横坐标位置为父结点的横坐标偏左一格，\出现的横坐标位置为父结点的横坐标偏右一格。
也就是说，如果树高为 m，则输出就有 2m-1 行。
第三部分为一个整数，表示将值代入变量之后，该中缀表达式的值。需要注意的一点是，除法代表整除运算，即舍弃小数点后的部分。

3.7.3 数据结构设计

```
struct TreeNode
{
    char val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(char v):val(v),left(nullptr),right(nullptr){};
};
```

3.7.4 功能说明（函数、类）

```
TreeNode* BuildExpressionTree(const string& expression)
{

    //用栈存储树的结点和操作符

    stack<TreeNode*>Tree;
    stack<char>ops;

    //编译表达式

    for(char c : expression)
    {
        if(isspace(c))
          continue;
        if(isalpha(c))
        {

            //当字符是操作数时，建立新结点并且入栈

            TreeNode* newNode=new TreeNode(c);
            Tree.push(newNode);
        }

        else if(c=='(')//左括号直接入栈
```

```
            ops.push(c);
        else if(c==')')
        {
            //和之前布尔表达式类似，遇到右括号则不断弹出操作符并建立子树，直到遇到左括号为止
            while(!ops.empty() && ops.top()!='(')
              makeSubTree(Tree,ops);
            ops.pop();
            //此处弹出左括号
        }
        else
        {
            //当入栈操作符优先级小于等于栈顶操作符时，不断弹出操作符并建立子树
            while(!ops.empty() && precedence(ops.top())>=precedence(c))
              makeSubTree(Tree,ops);
            ops.push(c);
        }
    }
    //将剩余操作符依次弹出并建立子树
    while(!ops.empty())
      makeSubTree(Tree,ops);
    //最终栈顶元素即为表达式树的根节点，返回根节点
    return Tree.top();
}

void printTree(TreeNode* root)
{
    if (!root) return;

    int h = getHeight(root);
    int maxWidth = (int)pow(2, h);
    //将每一行的内容视为一个字符串
    vector<string> canvas(2 * h - 1, string(maxWidth, ' '));
```

```
    int rootCol = maxWidth / 2-1;
    int gap = (int)pow(2, h - 2);

    //递归填充 canvas 矩阵，即每一行的内容

    fillMatrix(canvas, root, 0, rootCol, gap);

    for (string& line : canvas)
    {

        //去掉行末多余的空格，使用 string 类的 find_last_not_of 函数

        //此时从行末开始查找，查找第一个不是空格的位置

        int end = line.find_last_not_of(' ');
        if (end != string::npos)
            line = line.substr(0, end + 1);
        else
            line.clear();
        cout << line << endl;
    }
}

int evaluate(TreeNode*root,map<char,int>&valMap)
{

    //使用了 map 存储变量对应的值

    //操作符为 key 值，其真值为 value 值

    if(root==nullptr)
      return 0;
    if(isalpha(root->val))
      return valMap[root->val];
    int leftvalue=evaluate(root->left,valMap);
    int rightvalue=evaluate(root->right,valMap);
    switch(root->val)
    {
        case '+':
          return leftvalue+rightvalue;
        case '-':
          return leftvalue-rightvalue;
        case '*':
          return leftvalue*rightvalue;
        case '/':
          return leftvalue/rightvalue;
    }
```

```
}
```

3.7.5 调试分析（遇到的问题和解决方法）

初始版本的代码在打印树的结构时会发生结构重叠的问题，解决办法是 rootCol 和 gap 的初始值以及递归时的更新公式必须精确。
然后就是在 makeSubTree 函数中，我一开始的 pop 顺序写反了，写为了先 pop 左子树再 pop 右子树，这样会导致严重的错误，因为栈是一个后进先出的结构，左子树是先进的栈，右子树是后进的栈，所以应该先 pop 右子树再 pop 左子树。

3.7.6 总结和体会

这道题要求构建算术表达式树并实现后序遍历、可视化打印和求值三个功能，算法难度较高，综合考察了栈、树、递归等多个知识点。核心思路是使用双栈算法：操作数栈存储树节点，操作符栈根据优先级规则决定何时弹出并构建子树，遇到操作数直接入树栈，遇到左括号入操作符栈，遇到右括号弹出直到左括号，遇到普通操作符则比较优先级决定是否先弹出栈顶构建子树。算法时间复杂度为 O(n)包括构建、遍历、求值都是线性，printTree 虽然画布宽度是指数级但节点数仍为 n；空间复杂度 O(n)用于栈和树，printTree 额外需要 O(h*2^h)空间。实现中需要注意几个边界条件：空格跳过、空树处理、操作符栈空时不比较优先级、括号作为哨兵、行末空格去除确保输出美观。易错点主要有四个：一是 makeSubTree 中必须先 pop 右子树再 pop 左子树（栈的 LIFO 特性），顺序错误会导致整棵树镜像翻转；二是优先级比较必须用>=而非>，否则同级运算符的左结合性无法正确处理；三是 printTree 的位置计算公式（rootCol、gap 及其更新）必须精确，错误会导致树形错位。

3.实验总结

通过这个综合实验，系统性地掌握了二叉树的存储、遍历、转换、重建和应用五大核心能力。从基础的操作到复杂的算法设计，从理论分析到实践编码，形成了对树结构的深刻理解。特别是学会了：

1. **灵活选择数据结构**：根据问题特点选择最合适的树表示方法
2. **递归思维**：将复杂问题分解为子问题的递归思想
3. **问题转换**：将原问题转换为已知问题的建模能力
4. **栈和队列的深度应用**：理解这两种基础数据结构在树算法中的核心作用
5. **边界条件处理**：养成了严谨的边界考虑习惯
6. **调试与优化**：从时间空间复杂度分析到代码优化的完整思路